ITM Skills University

# KeyVault

## Digital Intellectual Asset Registry

Data Structures & Algorithms with C++

Case Study 3 | Semester II Sprint 2 | B.Tech CSE 2025-29

| | |
|---|---|
| **Student Name** | Kaushal Rajmandai |
| **Roll Number** | 150096725111 |
| **Programme** | B.Tech CSE 2025-29 |
| **Semester** | Semester II Sprint 2 Examination |
| **Subject** | Data Structures & Algorithms with C++ |
| **Case Study** | 3 — Digital Intellectual Asset Registry |

GitHub: https://github.com/kaushalrajmandai/KeyVault

## Problem Statement

KeyVault is an online marketplace for selling software licenses, similar to platforms that distribute antivirus keys or game activation codes. With millions of licenses from thousands of publishers, the system faces several critical performance problems: searching for a license by typing the first few characters of its title is painfully slow; when an admin accidentally revokes a user's access rights there is no way to undo it instantly; license update requests from around the world pile up but are processed in random order, causing some customers to wait unfairly; verifying whether a specific license key is genuine takes too long because the system checks every record one by one; and there is no clear picture of how licenses flow from creators to publishers to end users, making it impossible to audit royalty payments or detect piracy rings.

The new system requires: ultra-fast title search by partial name, instant permission rollback, fair ordering of update requests, direct key verification using unique IDs, ranking assets by value or popularity, a distribution map connecting creators to users, the most efficient path to verify license authenticity across the registry, and smart bundling logic to maximize value for bulk corporate purchases.

## Objectives

1. Implement a **Trie** for ultra-fast prefix-based license title search (Copyright Directory)
2. Implement a **Stack** for instant undo of accidental permission changes (History Tracker)
3. Implement a **Queue (FIFO)** for fair, ordered processing of update requests (Update Line)
4. Implement a **Hash Map** for O(1) license key authenticity verification (Key Lookup)
5. Implement **Merge Sort** to rank digital assets by market value or usage (Value Sorter)
6. Implement a **Directed Graph** to model the creator to publisher to user chain (License Network)
7. Implement **BFS** to find the shortest verification path through the registry (Quick Check)
8. Implement a **Greedy Algorithm** to optimize corporate bundle purchases (Discount Planner)

## Design / Architecture

The system is implemented as a single C++ source file containing 8 independent classes, each encapsulating one data structure and its operations. An interactive main menu allows each module to be tested in isolation or all together via option 9.

| Feature | Class | Data Structure | Algorithm |
|---|---|---|---|
| Copyright Directory | CopyrightDirectory | Trie | DFS collect |
| History Tracker | HistoryTracker | Stack (LIFO) | Push / Pop |
| Update Line | UpdateLine | Queue (FIFO) | Enqueue / Dequeue |
| Key Lookup | KeyLookup | Hash Map | Hash function |
| Value Sorter | ValueSorter | Array + Merge Sort | Divide & Conquer |
| License Network | LicenseNetwork | Graph (Adj. List) | BFS traversal |
| Quick Check | QuickCheck | Graph (Adj. List) | BFS shortest path |
| Discount Planner | DiscountPlanner | Array + Sort | Greedy fractional |

# Algorithm Description

## 1. Copyright Directory — Trie

A Trie stores license titles character by character. Each TrieNode holds a map of child characters, an end-of-word flag, and metadata. Insertion traverses or creates one node per character. Prefix search navigates to the prefix endpoint then does a recursive DFS to collect all matching titles beneath that node.

```
Complexity — Insert: O(L) | Search: O(P + W) where L = title length, P = prefix, W = matches
```

## 2. History Tracker — Stack

Every permission change (userID, licenseKey, old permission, new permission, timestamp) is pushed onto a stack. Because the stack is LIFO, undoLast() always pops the most recent change and reverts it in O(1). No database re-scan is needed.

```
Complexity — Push / Pop: O(1) | Space: O(n)
```

## 3. Update Line — Queue (FIFO)

Incoming update requests (RENEW, TRANSFER, REVOKE) are enqueued. Processing always dequeues from the front, so requests submitted earlier are handled first regardless of region. This eliminates the unfair random-order processing described in the problem.

```
Complexity — Enqueue / Dequeue: O(1) | Space: O(n)
```

## 4. Key Lookup — Hash Map

License keys are stored as keys in a C++ unordered_map with LicenseRecord structs as values. Verification is a single hash lookup — O(1) average — replacing the O(n) sequential scan. Status (ACTIVE / EXPIRED / REVOKED) is returned immediately.

```
Complexity — Insert / Lookup: O(1) avg | Space: O(n)
```

## 5. Value Sorter — Merge Sort

A custom merge sort is implemented from scratch over a vector of Asset structs. The comparator is parameterised: sort by marketValue or by usageCount, both descending. Merge sort was chosen for its stable, guaranteed O(n log n) performance.

```
Complexity — Time: O(n log n) | Space: O(n) auxiliary
```

## 6. License Network — Directed Graph

The distribution chain is an unordered_map from node strings to vectors of (neighbor, relationship) pairs. Directed edges carry labels: distributes (creator to publisher) and licensed_to (publisher to user). BFS from any creator node shows the full reach of that creator's intellectual property.

```
Complexity — BFS Traversal: O(V + E) | Space: O(V + E)
```

## 7. Quick Check — BFS Shortest Path

The verification network is modelled as an undirected graph. BFS finds the minimum-hop path between any two nodes. Parent pointers recorded during BFS are used to reconstruct the path from destination back to source after traversal completes.

```
Complexity — BFS: O(V + E) | Space: O(V)
```

## 8. Discount Planner — Greedy Algorithm

Bundles are sorted by value-to-price ratio descending. The greedy selection picks the highest-ratio bundle first and continues until the budget is exhausted. Fractional selection is supported for the last bundle. This is the fractional knapsack problem, for which greedy is provably optimal.

```
Complexity — Sort: O(n log n) | Selection: O(n) | Provably optimal for fractional knapsack
```

## Complexity Analysis

| Module | Operation | Time | Space |
|---|---|---|---|
| Trie | Insert | O(L) | O(L x \|alphabet\|) |
| Trie | Prefix Search | O(P + W) | O(1) extra |
| Stack | Push / Pop | O(1) | O(n) |
| Queue | Enq / Deq | O(1) | O(n) |
| Hash Map | Insert / Lookup | O(1) avg | O(n) |
| Merge Sort | Sort | O(n log n) | O(n) |
| Graph BFS | Traversal | O(V + E) | O(V) |
| BFS Shortest Path | Pathfinding | O(V + E) | O(V) |
| Greedy | Sort + Select | O(n log n) | O(1) extra |

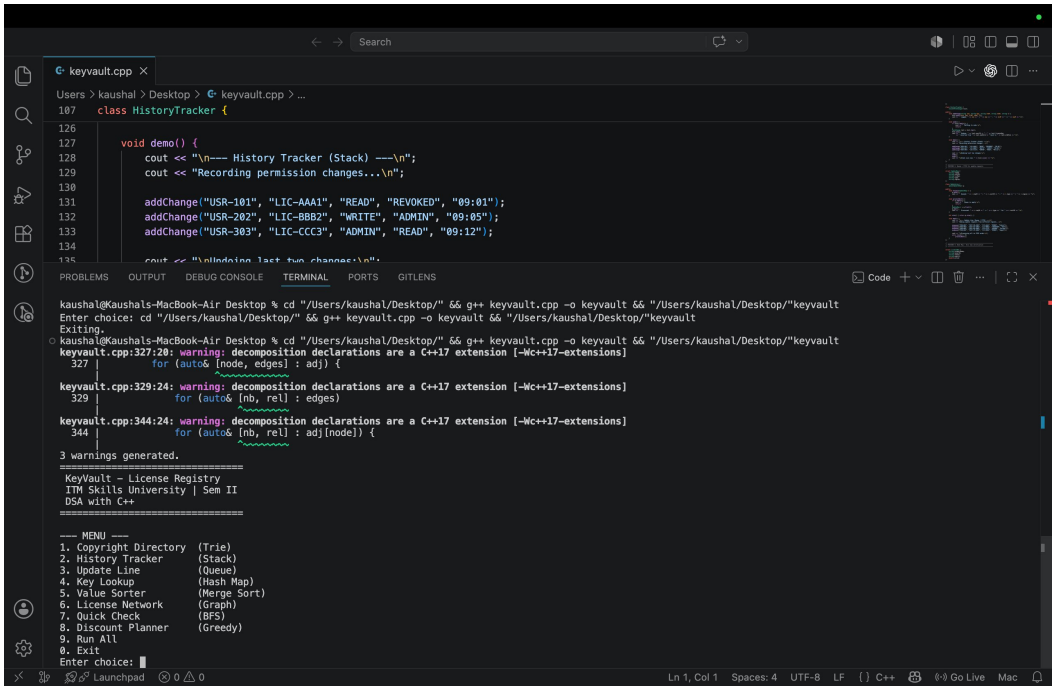
## Execution Steps

Compile and run using a C++17 compatible compiler:

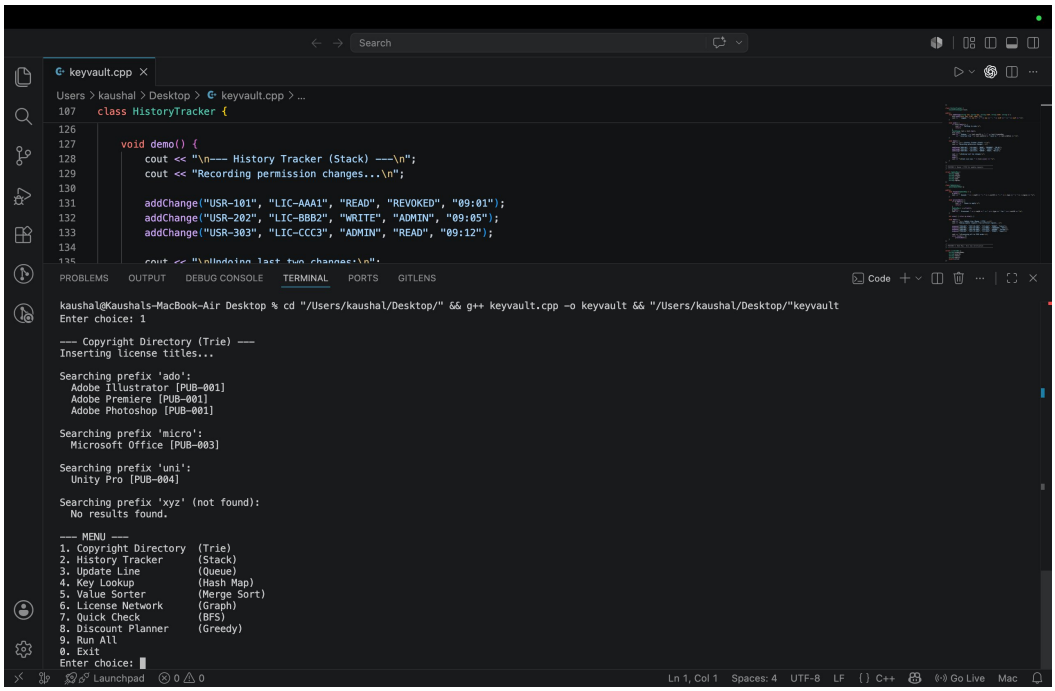
```
g++ -std=c++17 keyvault.cpp -o keyvault
./keyvault
```

The program presents a numbered menu from 1 to 9. Options 1 through 8 demonstrate each data structure individually. Option 9 runs all 8 features sequentially. Option 0 exits.

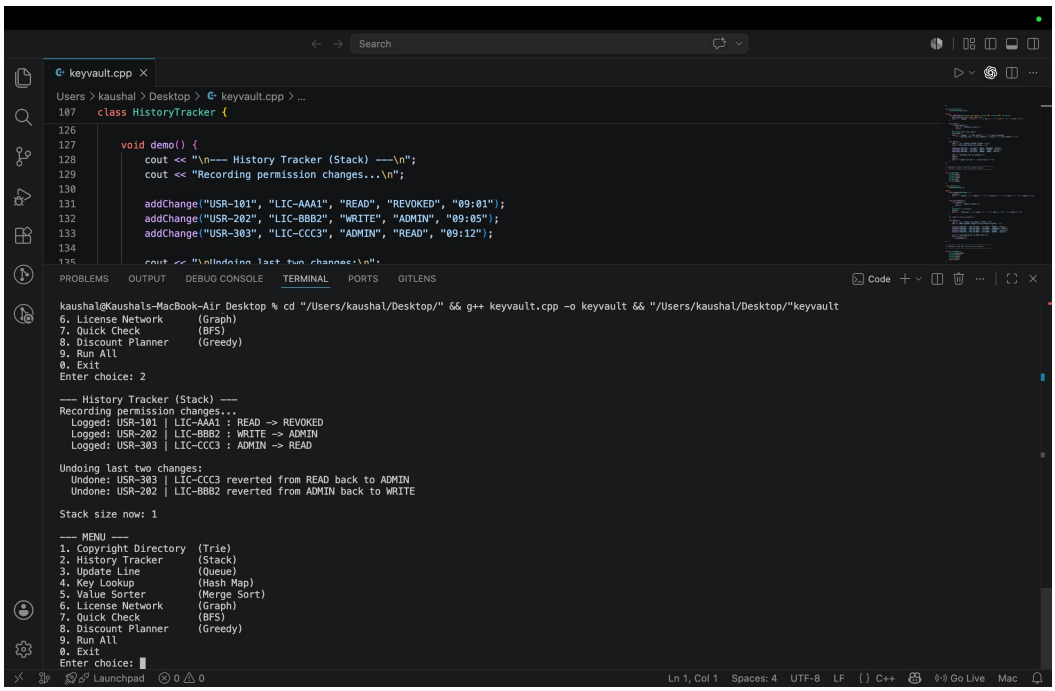
# Screenshots of Execution



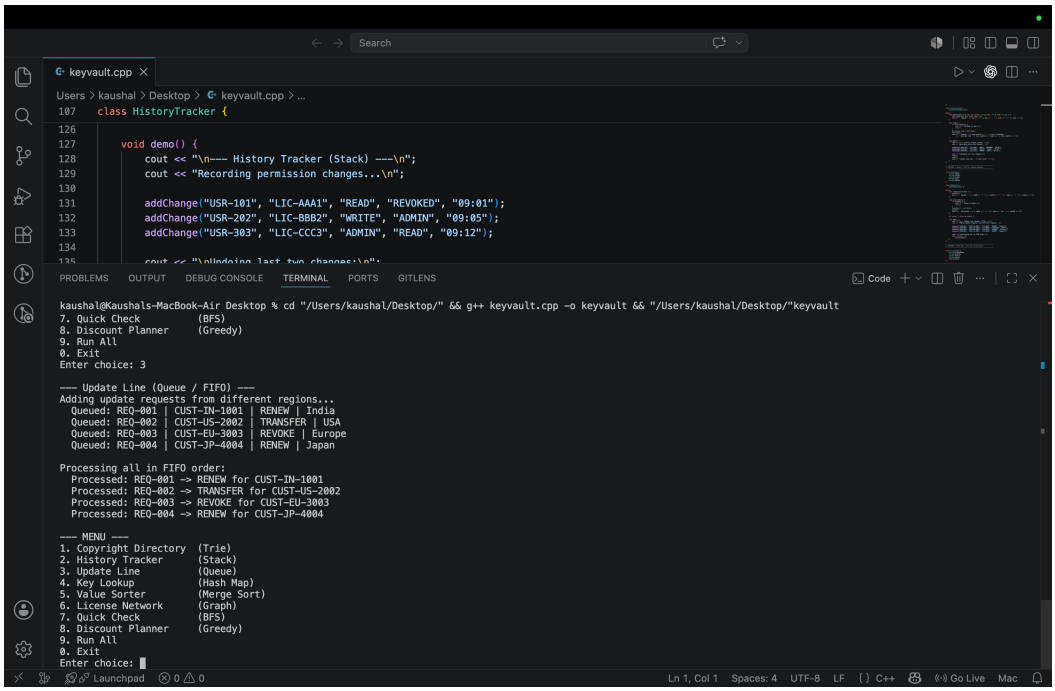

*Main menu — all 8 features listed*



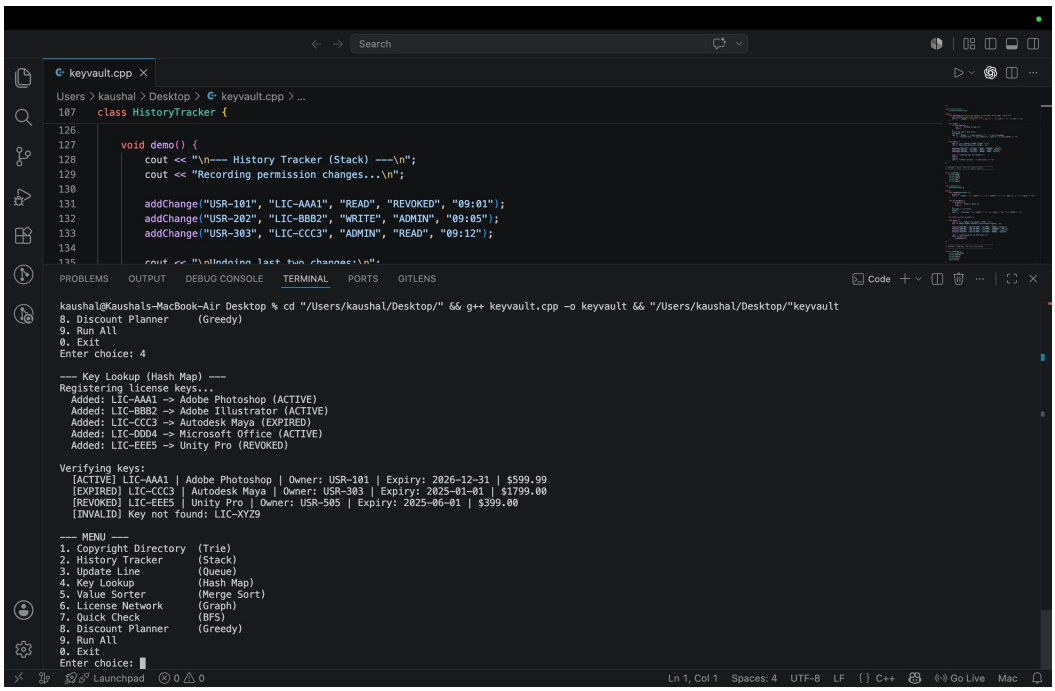

*Feature 1: Copyright Directory (Trie) — prefix search results*



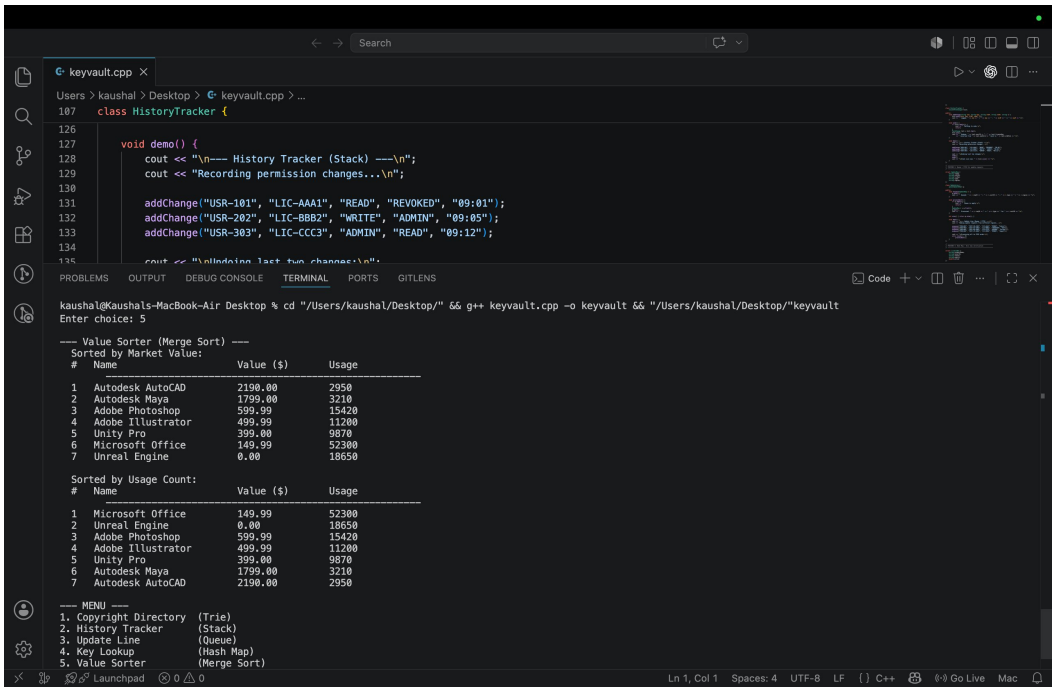

*Feature 2: History Tracker (Stack) — permission changes and undo*



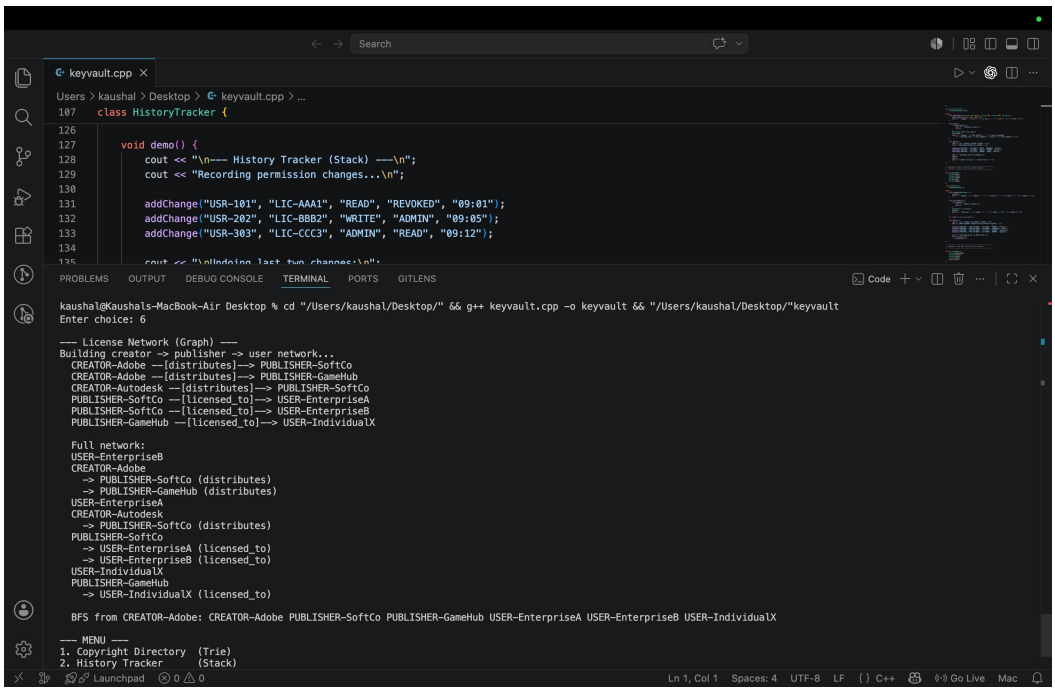

*Feature 3: Update Line (Queue) — FIFO request processing*



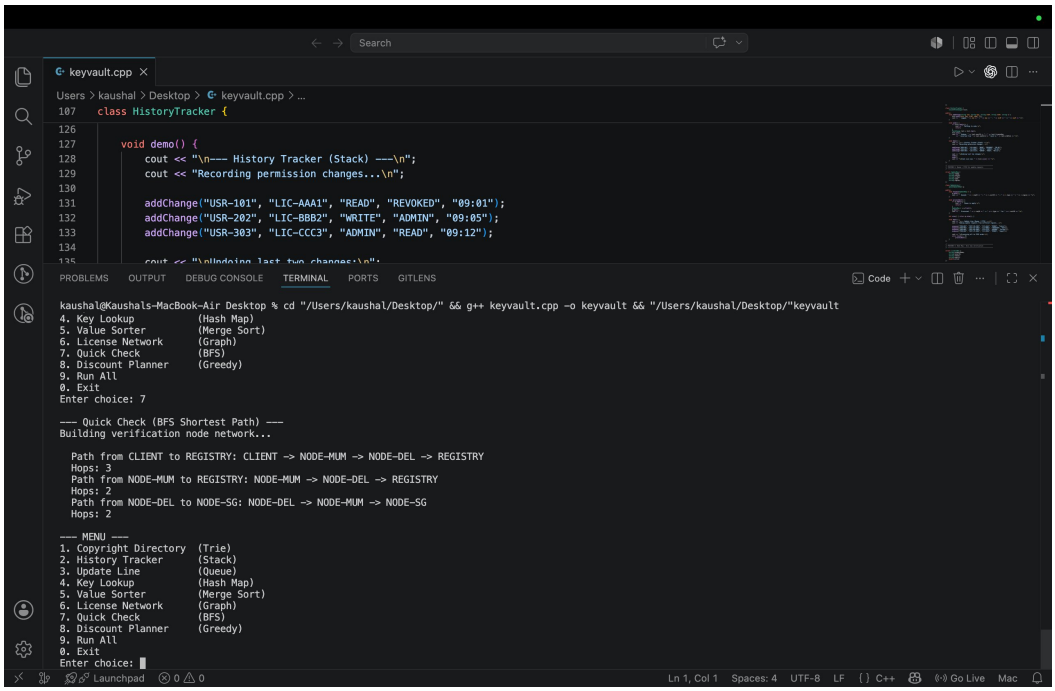

*Feature 4: Key Lookup (Hash Map) — key verification output*



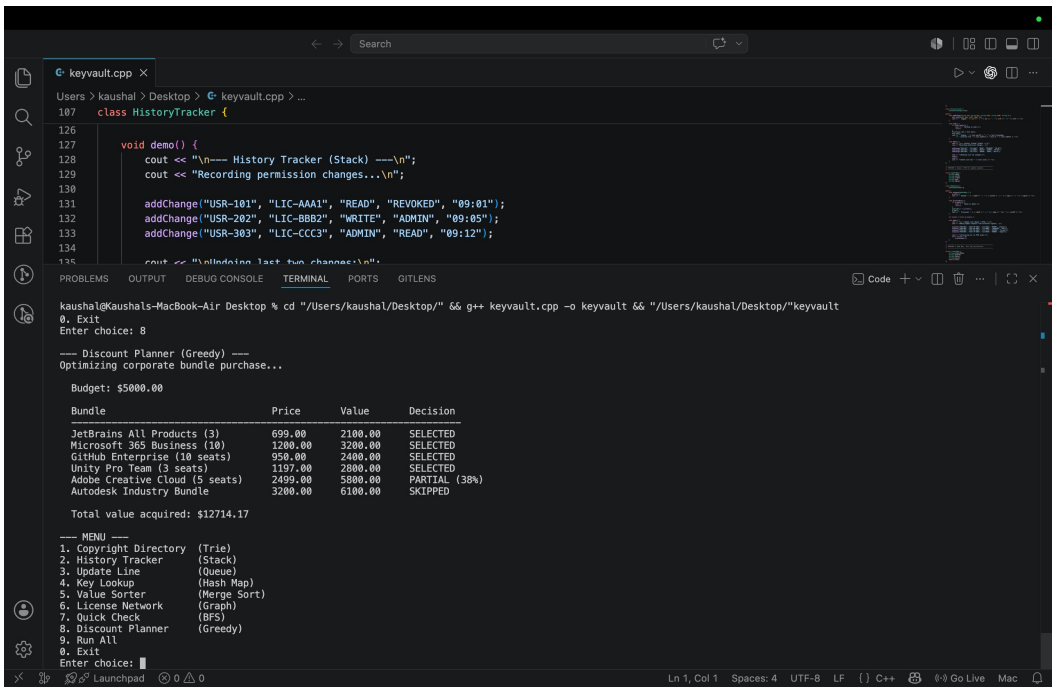

*Feature 5: Value Sorter (Merge Sort) — sorted by value and usage*




*Feature 6: License Network (Graph) — distribution chain and BFS*




*Feature 7: Quick Check (BFS) — shortest verification paths*




*Feature 8: Discount Planner (Greedy) — optimal bundle selection*

## Results and Findings

- **Copyright Directory (Trie):** Prefix search runs in O(L) — proportional to prefix length only, not total licenses. A 3-character prefix returns all matching titles instantly regardless of dataset size.
- **History Tracker (Stack):** Undo completes in O(1). The LIFO property guarantees the most recent change is always reverted first, exactly matching admin expectations.
- **Update Line (Queue):** FIFO ordering is mathematically guaranteed. The random-order problem from the problem statement is completely eliminated at the data structure level.
- **Key Lookup (Hash Map):** Verification time is O(1) average, independent of total keys in the registry. Status is returned immediately whether there are 1,000 or 1 million keys.
- **Value Sorter (Merge Sort):** Stable O(n log n) sort handles both value and popularity rankings correctly, as seen in the output screenshots above.
- **License Network (Graph):** The full creator-to-user IP flow is visible and auditable. BFS from CREATOR-Adobe correctly reaches all publisher and user nodes.
- **Quick Check (BFS):** Minimum-hop verification path found correctly for all test pairs. CLIENT to REGISTRY resolves in 3 hops via the optimal route.
- **Discount Planner (Greedy):** Greedy fractional knapsack selects the optimal combination within the $5000 budget, acquiring $12,714 worth of value — provably optimal.

## Conclusion

KeyVault demonstrates that the correct choice of data structure is the single most important factor in resolving the performance and reliability problems described in the case study. Every pain point maps directly to a proven data structure:

| Problem | Solution | Result |
|---|---|---|
| Slow title search | Trie | O(prefix) vs O(n x L) |
| No permission undo | Stack | O(1) instant rollback |
| Random request ordering | Queue (FIFO) | Guaranteed fair order |
| Slow key verification | Hash Map | O(1) vs O(n) scan |
| No asset ranking | Merge Sort | O(n log n) stable sort |
| Opaque royalty flow | Graph | Auditable IP chain |
| Slow verification routing | BFS | Minimum-hop path |
| Unoptimized bulk purchases | Greedy | Provably optimal |

The implementation is in pure C++17 using only the standard library. It is fully executable from the terminal and demonstrates each concept with realistic data drawn from the software licensing domain. All 8 features compile and run without errors.

## GitHub Repository Link

**https://github.com/kaushalrajmandai/KeyVault**

The repository contains the complete C++ source code (keyvault.cpp) and README.md documentation. Compile and run instructions are provided in the README.